
Gestion des rendus

4 janvier de 2026

AIT MOHAMED Younes (aitmohay)

BEAURAIN Tom (beurato)

BEN ISMAIL Wajd (benismaw)

FURLANI DA SILVA SOARES Manuela (furlanim)

MARIA ENGLER PINTO Carlos (mariaenc)

A) Synchronisation et état Git

- Se placer sur la branche à rendre (ex. `master`) et vérifier : `git branch --show-current`
- Mettre à jour : `git pull`
- Vérifier qu'il n'y a rien d'oublié : `git status` doit être **clean**
- Vérifier que tous les fichiers nouveaux sont ajoutés : `git status` ne doit pas afficher de "Untracked files"
- Vérifier l'historique : `git log -1` (identifier le commit exact du rendu)
- Pour éviter que la branche `master` n'ait pas un code fonctionnel, on va utiliser une branche parallèle à `main` qui s'appelle `develop`. On va faire le merge sur la branche `master` seulement les versions fonctionnelles du compilateur avant chaque rendu.
- On va utiliser

B) Build + tests automatisés (obligatoire)

- Compiler/tests : `mvn test` passe entièrement (BUILD SUCCESS)
- Vérifier que les scripts de tests fournis (dont `common-tests.sh`) ont bien été exécutés via `mvn test`
- (Optionnel mais conseillé) `mvn clean test` pour partir de zéro

C) Conformité de l'organisation des tests .deca

- Vérifier l'arborescence `src/test/deca/` :
 - `syntax/valid` et `syntax/invalid`
 - `context/valid` et `context/invalid`

- `codegen/valid` et `codegen/invalid`
- Vérifier la classification :
 - Étape A : `syntax/invalid` = erreurs lexicales/syntaxiques (ex. `decac -p` doit échouer)
 - Étape B : `context/invalid` = syntaxe correcte mais erreurs contextuelles
 - Étape C : `codegen/invalid` = exécution provoque une erreur (ex. division par zéro)
- Vérifier la règle **interactive** :
 - Tous les tests utilisant `readInt` / `readFloat` sont **uniquement** dans `src/test/deca/codegen/interactive/`
 - Aucun `readInt/readFloat` dans `codegen/valid` ni `codegen/invalid`

D) Smoke tests (rapide, mais très utile)

- Compiler et exécuter 2–3 programmes “vitrine” simples :
 - un programme sans erreur (print, calculs)
 - un programme qui déclenche une erreur attendue (ex. division par zéro en C)
 - un programme de contrôle de flux (if/while)
- Vérifier le format des messages d’erreur (fichier:ligne:colonne: ...) si demandé par la spec

E) Nettoyage et qualité du dépôt

- Aucun debug “bruyant” (prints oubliés), logs maîtrisés
- Les scripts de test sont exécutables (`chmod +x` si nécessaire) et versionnés
- Les documents exigés par le rendu (planning/docs) sont présents au bon endroit (si applicable au rendu intermédiaire)

F) Push et vérification sur le dépôt distant

- `git push` effectué
- Vérifier sur le dépôt distant que le dernier commit est bien celui du rendu
- (Recommandé) Test de reproductibilité : cloner dans un répertoire vierge et relancer `mvn test`

Extension ARM

ARM-1) Environnement / toolchain

- Vérifier la toolchain ARM (version et commandes disponibles), par ex. `arm-... --version`
- Vérifier la présence de QEMU/Docker/VM (selon votre choix) et la version si utile

ARM-2) Scripts uniques et reproductibles

- Un script unique (ex. `run_arm.sh`) permet de :
 - générer l'assembleur ARM (depuis `decac`)
 - assembler + linker
 - exécuter sous QEMU (ou environnement choisi)
 - comparer la sortie avec l'oracle (diff)

ARM-3) Smoke tests ARM (minimum)

- "Hello / print string" fonctionne
- Aritmétique + variables (résultat déterministe) fonctionne
- if/while (contrôle de flux) fonctionne
- (si applicable) tests d'erreurs runtime (ex. division par zéro) signalées correctement